COURSE CODE: DSA0101

COURSE: Object oriented programming with c++

STUDENT NAME: M.LAKSHMI

REGISTER NO: 1911260117

ASSIGNMENT REPORT

SMART HOSPITAL BED & PATIENT MONITORING MANAGEMENT SYSTEM

Using Parameterized Constructor in C++

1. AIM

To design and implement a menu-driven Smart Hospital Bed and Patient Monitoring Management System in C++ using Object-Oriented Programming concepts, with special emphasis on the parameterized constructor for initializing patient, bed, and admission objects with meaningful values.

2. OBJECTIVE

- To register and manage different categories of patients.
- To allocate hospital beds according to patient category.
- To record admission and monitoring/service information.
- To calculate patient charges based on patient type and service usage.
- To demonstrate the use of parameterized constructors in C++.
- To initialize objects directly with user-provided or predefined values using constructors.
- To apply encapsulation, inheritance and polymorphism along with constructors.

3. PROBLEM STATEMENT

A hospital has a limited number of beds and different categories of patients. Manual bed allocation, patient registration, monitoring and billing can be difficult and time-consuming. The proposed system automates patient registration, bed allocation, admission records, monitoring/service usage, billing and bed-utilization reporting. Parameterized constructors are used to initialize objects with the required data when they are created, making the program organized and easier to manage.

4. PARAMETERIZED CONSTRUCTOR USED

A parameterized constructor is a constructor that accepts arguments when an object is created. It is useful when an object must be initialized with specific values instead of default values. In this project, parameterized constructors are used in the Patient hierarchy, HospitalBed and Admission classes.

For example, the Patient class uses a constructor that accepts patient ID, name, age and diagnosis. A GeneralPatient, ICUPatient or EmergencyPatient object passes these values to the Patient constructor. The HospitalBed constructor receives the bed number and ward type, while the Admission constructor receives the patient pointer, bed number, number of days and service units.

This approach ensures that an object is initialized with valid information at the time of creation.

5. CLASS DETAILS

Class	Type	Responsibility
Patient	Base class	Stores common patient information and provides virtual functions.
GeneralPatient	Derived class	Provides general ward charges and standard monitoring.
ICUPatient	Derived class	Provides higher ICU charges and monitoring requirements.
EmergencyPatient	Derived class	Provides emergency charges and higher priority.
HospitalBed	Independent class	Maintains bed number, ward type and occupancy.
Admission	Independent class	Maintains patient-bed association, days, services and bill.
Hospital	Management class	Handles collections, allocation, reports and menu.

6. ALGORITHM / PSEUDOCODE

1. Start the program.

2. Create the Hospital object and initialize hospital beds using parameterized constructors.

3. Display the main menu.

4. Register a patient by entering ID, name, age, diagnosis and patient type.

5. Create the selected patient object using its parameterized constructor.

6. Store the patient object in the patient collection.

7. Display patients and bed availability when requested.

8. For admission, identify the patient's category and allocate a suitable free bed.

9. Create an Admission object using a parameterized constructor.

10. Calculate the current bill using patient-specific virtual functions.

11. Display admission records and compare admission bills.

12. Generate the bed-utilization report.

13. Repeat the menu until the user selects Exit.

14. Destroy the Hospital object and release dynamically allocated patients.

15. Stop.

7. C++ PROGRAM

```
#include <iostream>
#include <vector>
#include <iomanip>
using namespace std;

// Base class
class Patient {
protected:
    int patientId;
    string name;
    int age;
    string diagnosis;

public:
    Patient() : patientId(0), name(""), age(0), diagnosis("") {}

    // Parameterized constructor
    Patient(int id, string n, int a, string d)
        : patientId(id), name(n), age(a), diagnosis(d) {}

    virtual ~Patient() {}

    virtual double getDailyCharge() const = 0;
    virtual double getServiceCharge() const = 0;
    virtual int getPriority() const = 0;
    virtual string getType() const = 0;

    int getId() const { return patientId; }
    string getName() const { return name; }

    virtual void display() const {
        cout << "ID: " << patientId
             << ", Name: " << name
             << ", Age: " << age
             << ", Diagnosis: " << diagnosis << endl;
    }
};

// Derived class 1
class GeneralPatient : public Patient {
public:
    GeneralPatient() : Patient() {}

    // Parameterized constructor
    GeneralPatient(int id, string n, int a, string d)
        : Patient(id, n, a, d) {}
```

```
    double getDailyCharge() const override { return 2000; }
    double getServiceCharge() const override { return 300; }
    int getPriority() const override { return 1; }
    string getType() const override { return "General"; }
};

// Derived class 2
class ICUPatient : public Patient {
public:
    ICUPatient() : Patient() {}

    // Parameterized constructor
    ICUPatient(int id, string n, int a, string d)
        : Patient(id, n, a, d) {}

    double getDailyCharge() const override { return 7500; }
    double getServiceCharge() const override { return 1000; }
    int getPriority() const override { return 2; }
    string getType() const override { return "ICU"; }
};

// Derived class 3
class EmergencyPatient : public Patient {
public:
    EmergencyPatient() : Patient() {}

    // Parameterized constructor
    EmergencyPatient(int id, string n, int a, string d)
        : Patient(id, n, a, d) {}

    double getDailyCharge() const override { return 5000; }
    double getServiceCharge() const override { return 750; }
    int getPriority() const override { return 3; }
    string getType() const override { return "Emergency"; }
};

// Hospital Bed
class HospitalBed {
private:
    int bedNumber;
    string wardType;
    bool occupied;

public:
    // Parameterized constructor
    HospitalBed(int no, string ward)
        : bedNumber(no), wardType(ward), occupied(false) {}

    bool isAvailable() const { return !occupied; }
    void occupy() { occupied = true; }
    void release() { occupied = false; }

    int getBedNumber() const { return bedNumber; }
    string getWardType() const { return wardType; }

    void display() const {
        cout << "Bed " << bedNumber
             << " | Ward: " << wardType
             << " | " << (occupied ? "Occupied" : "Available")
             << endl;
    }
};
```

```
// Admission class
class Admission {
private:
    Patient* patient;
    int bedNumber;
    int days;
    int serviceUnits;

public:
    // Parameterized constructor with default values
    Admission(Patient* p, int bed, int d = 1, int s = 0)
        : patient(p), bedNumber(bed), days(d), serviceUnits(s) {}

    double calculateBill() const {
        return patient->getDailyCharge() * days +
               patient->getServiceCharge() * serviceUnits;
    }

    int getBedNumber() const { return bedNumber; }

    bool operator>(const Admission& other) const {
        return calculateBill() > other.calculateBill();
    }

    friend ostream& operator<<(ostream& out, const Admission& a) {
        out << left << setw(12) << a.patient->getType()
            << setw(18) << a.patient->getName()
            << setw(8) << a.bedNumber
            << setw(8) << a.days
            << setw(12) << a.serviceUnits
            << fixed << setprecision(2)
            << a.calculateBill();
        return out;
    }
};

// Hospital management
class Hospital {
private:
    vector<Patient*> patients;
    vector<HospitalBed> beds;
    vector<Admission> admissions;

public:
    Hospital() {
        beds.push_back(HospitalBed(101, "General"));
        beds.push_back(HospitalBed(102, "General"));
        beds.push_back(HospitalBed(201, "ICU"));
        beds.push_back(HospitalBed(202, "ICU"));
        beds.push_back(HospitalBed(301, "Emergency"));
    }

    ~Hospital() {
        for (Patient* p : patients)
            delete p;
    }

    void registerPatient() {
        int id, age, type;
        string name, diagnosis;

        cout << "Enter Patient ID: ";
        cin >> id;
        cin.ignore();
```

```
    cout << "Enter Name: ";
    getline(cin, name);

    cout << "Enter Age: ";
    cin >> age;
    cin.ignore();

    cout << "Enter Diagnosis: ";
    getline(cin, diagnosis);

    cout << "1. General\n2. ICU\n3. Emergency\n";
    cout << "Enter Patient Type: ";
    cin >> type;

    Patient* p = nullptr;

    if (type == 1)
        p = new GeneralPatient(id, name, age, diagnosis);
    else if (type == 2)
        p = new ICUPatient(id, name, age, diagnosis);
    else if (type == 3)
        p = new EmergencyPatient(id, name, age, diagnosis);
    else {
        cout << "Invalid type.\n";
        return;
    }

    patients.push_back(p);
    cout << "Patient registered successfully.\n";
}

Patient* findPatient(int id) {
    for (Patient* p : patients)
        if (p->getId() == id)
            return p;
    return nullptr;
}

void displayPatients() const {
    cout << "\n--- All Patients ---\n";
    for (Patient* p : patients)
        p->display();
}

void displayBeds() const {
    cout << "\n--- Bed Status ---\n";
    for (const auto& bed : beds)
        bed.display();
}

void admitPatient() {
    int id;
    cout << "Enter Patient ID: ";
    cin >> id;

    Patient* p = findPatient(id);
    if (!p) {
        cout << "Patient not found.\n";
        return;
    }

    string requiredWard = p->getType();

    for (auto& bed : beds) {
```

```
        if (bed.isAvailable() &&
            bed.getWardType() == requiredWard) {

            bed.occupy();
            admissions.push_back(
                Admission(p, bed.getBedNumber())
            );

            cout << "Patient admitted to Bed "
                 << bed.getBedNumber() << ".\n";
            return;
        }
    }

    cout << "No suitable bed available.\n";
}

void displayAdmissions() const {
    cout << "\n--- Admission Records ---\n";
    cout << left << setw(12) << "Type"
         << setw(18) << "Patient"
         << setw(8) << "Bed"
         << setw(8) << "Days"
         << setw(12) << "Services"
         << "Bill\n";

    for (const auto& a : admissions)
        cout << a << endl;
}

void compareAdmissions() const {
    if (admissions.size() < 2) {
        cout << "At least two admission records are required.\n";
        return;
    }

    if (admissions[0] > admissions[1])
        cout << "First admission has a higher bill.\n";
    else
        cout << "Second admission has a higher or equal bill.\n";
}

void utilizationReport() const {
    int occupied = 0;

    for (const auto& bed : beds)
        if (!bed.isAvailable())
            occupied++;

    double utilization =
        (static_cast<double>(occupied) / beds.size()) * 100;

    double revenue = 0;
    for (const auto& a : admissions)
        revenue += a.calculateBill();

    cout << "\n--- Bed Utilization Report ---\n";
    cout << "Total Beds: " << beds.size() << endl;
    cout << "Occupied Beds: " << occupied << endl;
    cout << "Available Beds: "
         << beds.size() - occupied << endl;
    cout << "Utilization: " << fixed << setprecision(2)
         << utilization << "%\n";
    cout << "Revenue: Rs. " << revenue << endl;
```

```
    }

    void menu() {
        int choice;

        do {
            cout << "\n===== SMART HOSPITAL MANAGEMENT =====\n";
            cout << "1. Register Patient\n";
            cout << "2. Display All Patients\n";
            cout << "3. Display All Beds\n";
            cout << "4. Admit Patient / Allocate Bed\n";
            cout << "5. Display Admission Records\n";
            cout << "6. Compare Two Admissions\n";
            cout << "7. Generate Bed Utilization Report\n";
            cout << "8. Exit\n";
            cout << "Enter choice: ";
            cin >> choice;

            switch (choice) {
                case 1: registerPatient(); break;
                case 2: displayPatients(); break;
                case 3: displayBeds(); break;
                case 4: admitPatient(); break;
                case 5: displayAdmissions(); break;
                case 6: compareAdmissions(); break;
                case 7: utilizationReport(); break;
                case 8: cout << "Exiting...\n"; break;
                default: cout << "Invalid choice.\n";
            }
        } while (choice != 8);
    }
};

int main() {
    Hospital hospital;
    hospital.menu();
    return 0;
}
```

8. SAMPLE INPUT

1. Register Patient
Enter Patient ID: 101
Enter Name: Arun
Enter Age: 45
Enter Diagnosis: Fever
1. General
2. ICU
3. Emergency
Enter Patient Type: 1
Patient registered successfully.

4. Admit Patient / Allocate Bed
Enter Patient ID: 101

9. SAMPLE OUTPUT

Patient admitted to Bed 101.

--- Bed Status ---
Bed 101 | Ward: General | Occupied
Bed 102 | Ward: General | Available
Bed 201 | Ward: ICU | Available
Bed 202 | Ward: ICU | Available
Bed 301 | Ward: Emergency | Available

--- Admission Records ---

Type	Patient	Bed	Days	Services	Bill
General	Arun	101	1	0	2000.00

--- Bed Utilization Report ---
Total Beds: 5
Occupied Beds: 1
Available Beds: 4
Utilization: 20.00%
Revenue: Rs. 2000.00

10. BILLING RULES

Patient Type	Daily Bed Charge	Service/Monitoring Charge
General Patient	Rs. 2,000/day	Rs. 300/unit
ICU Patient	Rs. 7,500/day	Rs. 1,000/unit
Emergency Patient	Rs. 5,000/day	Rs. 750/unit

11. OOP CONCEPTS USED

- Encapsulation – private/protected data members and public member functions.
- Parameterized Constructor – initializes objects with required values at the time of object creation.
- Hierarchical Inheritance – GeneralPatient, ICUPatient and EmergencyPatient inherit from Patient.
- Polymorphism – virtual functions provide different behaviour for each patient category.
- Destructor – dynamically allocated Patient objects are released by Hospital.
- Operator Overloading – operator> compares admission bills and operator<< formats reports.
- Collections – vectors are used to manage patients, beds and admissions.

12. EXPLANATION OF PARAMETERIZED CONSTRUCTOR

The parameterized constructor is the central concept demonstrated in this assignment. When a patient object is created, values such as patient ID, name, age and diagnosis are supplied as arguments. These arguments are passed from the derived-class constructor to the base-class Patient constructor through the member-initializer list.

For example, GeneralPatient(id, name, age, diagnosis) creates a GeneralPatient object and passes the patient information to Patient(id, name, age, diagnosis). Similarly, HospitalBed(no, ward) initializes the bed number and ward type, and Admission(p, bed, d, s) initializes an admission record. Therefore, parameterized constructors reduce repetitive initialization code and ensure that each object starts with the information required by the system.

13. RESULT

Thus, the Smart Hospital Bed and Patient Monitoring Management System was designed and implemented successfully in C++ with parameterized constructors. The system demonstrates how constructors can initialize patient, bed and admission objects while working together with encapsulation, hierarchical inheritance, polymorphism, destructor, operator overloading and collections.

14. REFLECTION

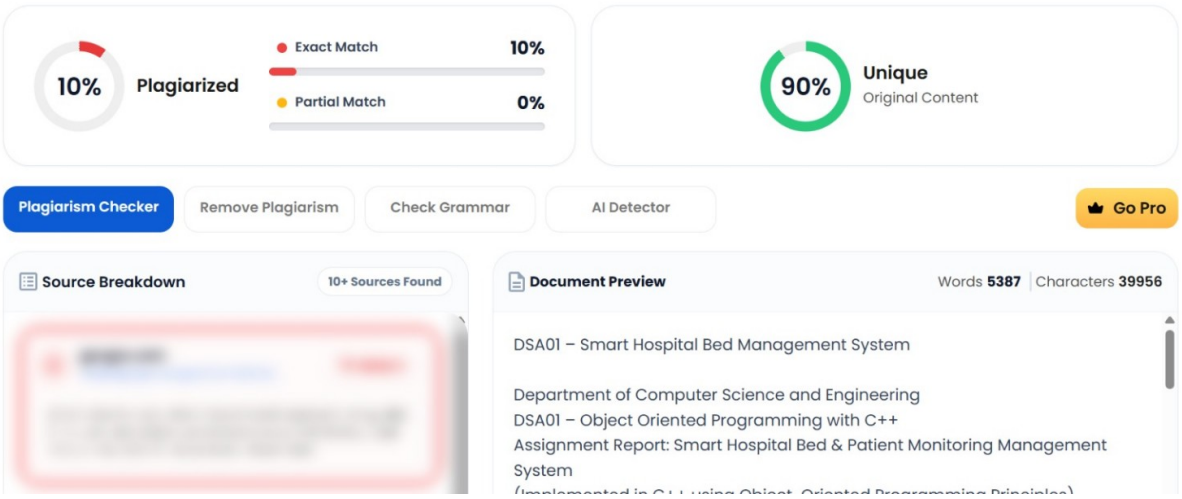
The parameterized constructor is useful in this system because different objects require different initial values. Patient details, bed details and admission details can be supplied directly when objects are created. This makes object initialization clear, structured and efficient. The concept also works naturally with inheritance because derived-class constructors can invoke the parameterized constructor of the base class.

15. SDG MAPPING

- SDG 3 – Good Health and Well-being: The system supports organized patient admission, monitoring and hospital-bed management.
- SDG 9 – Industry, Innovation and Infrastructure: The system demonstrates an automated software solution for improving hospital infrastructure and workflow.

16. CONCLUSION

The proposed Smart Hospital Bed and Patient Monitoring Management System reduces manual work in hospital bed and patient management. The use of parameterized constructors provides an efficient way to initialize objects with meaningful data at creation time. Combined with object-oriented programming concepts, the menu-driven implementation provides a structured and practical approach for managing patients, beds, admissions, billing and utilization reports.


10%
Plagiarized
Exact Match
10%
Partial Match
0%
90%
Unique
Original Content
Plagiarism Checker
Remove Plagiarism
Check Grammar
AI Detector
Go Pro
Source Breakdown
10+ Sources Found
Document Preview
Words 5387
Characters 39956
DSA01 – Smart Hospital Bed Management System
Department of Computer Science and Engineering
DSA01 – Object Oriented Programming with C++
Assignment Report: Smart Hospital Bed & Patient Monitoring Management System